# Autonomous Articulated Vehicle Control via Deep Reinforcement Learning and High-Dimensional Perception

Benjamin Schnapp

July 8, 2026

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Motivation

*[Section content omitted for review.]*

## 1.2 Problem Definition

*[Section content omitted for review.]*

## 1.3 Thesis Contributions

*[Section content omitted for review.]*

## 1.4 Institutional Context

*[Section content omitted for review.]*

Table 1.1: MASc thesis requirements summary

| **[Omitted for review]** |
| --- |
| Table content omitted for review. |

## 1.5 Thesis Organization

*[Section content omitted for review.]*

# Chapter 2

# Literature Review

Controlling an autonomous tractor-trailer draws on two methodological traditions: classical model-based control of articulated vehicles, and learning-based planning and control that operates on high-dimensional perception. This chapter reviews both, together with the sim-to-real transfer and safety considerations that bound any physical deployment. The account is organized as a single argument rather than an exhaustive catalogue. It begins with the control problem that makes articulated vehicles distinctive, surveys the classical methods that have long addressed it and the learning-based methods that now challenge them, examines how perception, the simulation-to-reality gap, and the governing safety requirements shape a learned controller, and closes by identifying the specific gap that this thesis fills.

## 2.1 The Articulated-Vehicle Control Problem

Autonomous driving is conventionally decomposed into a pipeline of perception, planning, and control, with each stage feeding the next [1]. Within that pipeline the tractor-trailer is a demanding case for the control stage, because of the passive hitch that couples its two bodies.

Prior analyses have established why. The hitch adds an internal configuration variable, the articulation angle, whose reverse dynamics are open-loop unstable. In forward motion a small hitch disturbance decays as the tractor pulls the trailer back into alignment, but in reverse the sign of the longitudinal velocity flips this behaviour: a small articulation error grows rather than decays and, without corrective steering, runs away to the jackknife condition, a folded configuration from which no steering input can recover the trailer [2]. Reverse driving is additionally non-minimum-phase, in that to move the trailer to one side the tractor must first steer to the other [3]. These two properties, open-loop instability and non-minimum-phase response, are what separate articulated control from ordinary rigid-body lane keeping. The kinematic model that produces them is derived in chapter 3,

and the confined low-speed setting that fixes the capabilities a controller must deliver is set out in chapter 1.

## 2.2 Classical Model-Based Control

The longest-established response to articulated instability is model-based control: derive a dynamic model, then design a control law that provably stabilizes it. This section reviews the two layers of that approach, the feedback controllers that stabilize and track, and the motion planners that supply the reference for them to track. Together they form the conventional pipeline against which the learned controller of this thesis is benchmarked.

### 2.2.1 Feedback Control and Stabilization

At the simplest level, a proportional-integral-derivative (PID) controller is trivial to implement and computationally negligible, but it acts only on the present error and cannot anticipate the jackknife condition; its gains must also be re-tuned for different payloads and path curvatures. A linear state-feedback law derived from the linearized tractor-trailer model addresses the multi-variable coupling more directly. Choosing that feedback as a linear-quadratic regulator (LQR) trades tracking accuracy against control effort through a quadratic cost and is optimal for the linearized plant [4]. The author's prior work applied exactly this design to the linearized tractor-trailer model [5], and an LQR/MPC tracker reappears as the conventional baseline in the deployment of chapter 6, with its formulation detailed in appendix A.

Model predictive control (MPC) generalizes linear feedback by re-optimizing the steering sequence over a receding horizon subject to explicit state and input constraints [6]. This constraint handling is valuable for articulated vehicles because a hard bound can be placed directly on the articulation angle. Linear MPC suffices near an operating point, whereas nonlinear MPC (NMPC) retains the full nonlinear kinematics, including the reverse instability, at a higher computational cost [7]. Control tailored specifically to the articulated case has taken two further forms. Active trailer steering adds an actuated degree of freedom at the trailer axle or hitch to damp articulation directly, and has been shown to improve both the high-speed stability and the low-speed manoeuvrability of tractor-semitrailers [8, 9]. For a conventional trailer with no such actuation, all authority rests on the tractor's steering angle, and recent work has used an intrinsically stable MPC formulation to suppress jackknifing while tracking a reference, exploiting stable inversion of the non-minimum-phase dynamics [10].

### 2.2.2 Geometric and Nonlinear Control

A parallel line of work exploits the geometric structure of the non-holonomic system directly. Trailer systems are a classic example of a differentially flat system, meaning that the full state and the control inputs can be written algebraically in terms of a flat output and its derivatives, which reduces trajectory generation to the problem of shaping that output [11]. Converting the kinematics to chained form enables time-varying feedback laws with provable path-following and point-stabilization guarantees for wheeled vehicles [12]. For the specific and difficult case of reverse motion, feedback schemes have been derived that stabilize a truck and trailer backing along a path in spite of the open-loop instability [3]. These methods achieve strong guarantees by committing fully to an analytical model of the vehicle, which is precisely the commitment that the learned controller of this thesis seeks to avoid.

### 2.2.3 Motion Planning for Articulated Vehicles

The feedback controllers above track a reference path, which must first be planned. For articulated vehicles this planning problem is itself difficult, because a feasible path must respect the vehicle's non-holonomic constraints and avoid configurations from which the trailer cannot recover. Sampling-based planners such as the rapidly-exploring random tree and its asymptotically optimal variant RRT$^*$ explore the configuration space without an explicit discretization [13, 14], whereas search-based freespace planners such as hybrid A$^*$ discretize the search yet retain kinematic feasibility [15]. Tailored to the articulated case, lattice-based planners assemble precomputed motion primitives that connect circular-equilibrium configurations, enabling complex forward and reverse manoeuvres [16], and optimization-based formulations compute reference paths that minimize the swept area of the vehicle bodies subject to road and obstacle constraints [17]. Once a path exists, geometric trackers such as pure pursuit follow it by steering toward a look-ahead point on the path [18], and refined trackers such as the Stanley controller add an explicit cross-track error term with demonstrated performance on full-scale autonomous vehicles [19].

### 2.2.4 Strengths and Limitations

The appeal of this pipeline is that it comes with guarantees: given an accurate model, the stability of the controller and the feasibility of the planned path can be analyzed and, in principle, guaranteed. Its cost is precisely that dependence. The model must be identified and kept accurate across payloads and operating conditions, the controllers and planners must be tuned, and the system must be assembled from separately designed planning and control stages. These costs motivate the alternative pursued in this thesis: a controller learned directly from interaction, which maps observations to steering commands without

an explicit model inversion or a separately planned reference trajectory, though the path to be followed still enters through the policy's observation, and which is benchmarked in chapter 5 against tuned instances of exactly the classical controllers reviewed here.

## 2.3 Learning-Based Control

Where a classical controller encodes the vehicle model explicitly and executes a fixed control law, a reinforcement learning (RL) agent infers a control policy directly from interaction with its environment, optimizing the policy end-to-end against a scalar reward [20]. This trades the analytical guarantees of the model-based approach for the ability to learn behaviours that are difficult to hand-design and to fold perception and control into a single learned mapping. This section traces the RL methods relevant to the thesis from their general foundations to their specific application to articulated vehicles.

### 2.3.1 Deep Reinforcement Learning Foundations

Modern RL for control rests on the deep-network function approximators that made high-dimensional value estimation practical. The Deep Q-Network first showed that a single architecture could learn to act directly from high-dimensional visual input across a range of tasks, using experience replay and a target network to stabilize learning [21]. Value-based methods of this kind are naturally suited to discrete action sets; the continuous steering and speed commands of a vehicle instead call for policy-based methods.

### 2.3.2 Actor-Critic and Policy-Gradient Methods

The deterministic policy gradient provides a route to continuous control by learning a deterministic actor alongside a critic that estimates action value [22]. Deep Deterministic Policy Gradient (DDPG) is the deep-network realization of this idea and a canonical off-policy actor-critic algorithm for continuous action spaces [23], but it is prone to critic overestimation bias and unstable updates. Two lines of work address this. Proximal Policy Optimization (PPO) is an on-policy method that improves stability through a clipped objective, at the cost of lower sample efficiency, which matters when simulation rollouts are expensive [24]. Soft Actor-Critic (SAC) instead augments the reward with an entropy term to encourage exploration and improve robustness in continuous control [25]. This thesis adopts Twin Delayed DDPG (TD3), a direct successor to DDPG that targets its two weaknesses through twin critics, delayed actor updates, and target-policy smoothing [26], and whose role is detailed in chapter 4. The author's prior coursework used DDPG for single-body path following [27]; the move to TD3 here is a direct response to the training instability observed in that work.

### 2.3.3 Reinforcement Learning for Autonomous Driving

RL has been applied across the driving stack, and recent surveys catalogue its use for lane keeping, behavioural decision making, and end-to-end control [28]. Two threads are relevant here. The first is end-to-end learning, in which a single network maps sensor input to control, whether trained by imitation of a human driver [29] or by reinforcement; a notable demonstration trained a lane-following policy on a full-scale car with on-board RL in a single day [30]. The second is the explicit framing of the driving task as a Markov decision process with a shaped reward, as set out in early RL-for-driving frameworks [31]. Both threads inform the observation and reward design developed in this thesis.

### 2.3.4 Reinforcement Learning for Articulated Vehicles

Applying RL specifically to articulated vehicles is more recent but growing, and the earliest instance long predates deep RL: the neural truck-backer-upper of Nguyen and Widrow learned to reverse a trailer toward a target, and depended on training that exposed the network to the unstable configurations it had to correct [32]. Modern work has revisited the problem with deep methods. A DDPG backing controller paired with a fuzzy-logic supervisor learned to reverse while avoiding the jackknife state [33], and policy-gradient agents have been trained to track reference paths with a tractor-trailer using rewards that penalize both tracking deviation and articulation growth [34]. A recurring theme in this small body of work, and one this thesis examines directly through its reward and observation ablations, is the importance of the hitch-stability term: the reward must discourage articulation growth for reverse driving to be learned reliably. Reward shaping of this kind rests on a firm theoretical footing, as potential-based shaping can accelerate learning while provably preserving the optimal policy [35].

### 2.3.5 Sequence Models and Curriculum Learning

Beyond actor-critic control, Decision Transformers recast RL as sequence modelling, predicting an action from a context window of past states, actions, and returns-to-go [36]. Such models can represent longer temporal dependencies than a shallow actor-critic policy, but their performance depends heavily on the diversity of the offline dataset, and a dataset dominated by nominal tracking behaviour tends to give poor recovery from the high-error states it underrepresents, an instance of the distribution-shift problem that motivates deliberately collecting such states. This need for coverage of hard configurations reappears as a training-data problem regardless of the policy class, and it motivates curriculum learning: presenting tasks in a deliberately ordered sequence, or automatically emphasizing the situations the agent handles worst [37, 38]. This thesis retains an actor-critic formulation but adopts a failure-replay curriculum (chapter 4) that revisits

failed scenarios, an idea related in spirit to prioritized experience replay [39].

### 2.3.6 Learning under Actuation Delay

A policy trained against an idealized, instantaneous actuator can degrade when deployed on hardware whose steering and drive systems exhibit dead time and first-order lag. A constant actuation delay violates the Markov property of the underlying decision process, turning it into a delayed MDP whose optimal policy depends on the recent action history rather than the instantaneous state alone [40]. The learning literature offers two broad remedies: augmenting the state with a buffer of in-flight actions so that the delayed problem is again Markovian, and correcting the off-policy value estimates for the delay, as in the Delay-Correcting Actor-Critic [41]. This thesis instead takes a complementary, model-based route at deployment, namely a Smith-predictor compensator [42] that presents the delay-free-trained policy with a delay-compensated state estimate (chapter 6), which avoids retraining the policy against the physical delay.

## 2.4 Perception and Representation Learning

A learned controller is only as good as what it perceives. A central question of this thesis is whether a compact, hand-engineered state vector or a learned representation of a bird's-eye-view (BEV) image yields better and more transferable control. This section reviews the perception building blocks that make the latter possible.

### 2.4.1 Convolutional Networks for Driving

Convolutional neural networks (CNNs) are the standard front-end for visual perception, learning spatial features directly from images rather than from hand-specified descriptors [43, 44]. Network depth was later shown to be trainable at scale through residual connections, which underpin most modern vision backbones [45]. In driving, CNNs have been trained end-to-end to map camera images directly to steering commands [46]. The bird's-eye view is a particularly convenient representation for ground-vehicle control because it places the vehicle and its surroundings in a metric top-down frame; recent perception systems learn to lift camera images into exactly such a representation [47], and the classical occupancy grid is its long-standing hand-built analogue [48]. In this thesis a CNN processes a rendered BEV image to extract lane geometry and obstacle layout.

### 2.4.2 Representation Learning and Autoencoders

Training a control policy directly on raw images is sample-inefficient, because the policy must learn to see and to act at the same time. Autoencoders decouple these by com-

pressing images into a compact latent code in an unsupervised pre-training stage [49], an idea applied to RL to shrink the policy's input and accelerate learning [50]. Variational autoencoders impose a probabilistic structure on the latent space that can improve its regularity [51], and spatial autoencoders tailored to control learn latent features corresponding to task-relevant image locations [52]. The U-Net architecture, originally developed for image segmentation, preserves multi-scale spatial detail through skip connections and provides a stronger structural prior than a plain convolutional encoder [53]. This thesis treats these learned encoders as one end of a representation spectrum, with the engineered state vector at the other, and compares them directly in chapter 4.

### 2.4.3 Range Sensing and Point-Cloud Perception

Bird's-eye imagery is not the only high-dimensional modality available to a ground vehicle. Planar lidar provides a direct geometric measurement of the surrounding free space, which the classical occupancy grid represents as a discretized map of obstacle probability [48]. At the other extreme of complexity, deep networks such as PointNet learn features directly from raw, unordered three-dimensional point clouds [54]. This thesis deliberately adopts a lighter-weight lidar representation that sits between these poles: a fixed-length vector of range returns appended to the state vector, rather than a learned point-cloud encoder. This keeps the observation compact and the sensing model simple, and it allows the lidar and image modalities to be compared on an equal footing in chapter 4.

## 2.5 Sim-to-Real Transfer

Because the controllers in this thesis are trained entirely in simulation, transferring a simulation-trained policy to physical hardware is a first-class concern rather than an afterthought. The central obstacle is the reality gap, the systematic mismatch between simulation and hardware that degrades a naively transferred policy [55]. It is useful to separate this gap into a visual component, a difference in what the sensors report, and a dynamics component, a difference in how the vehicle responds to a command, because the two are attacked by different tools.

The dominant learning-based remedy is domain randomization, which randomizes parameters during training so that the real system appears to the policy as one more variation of a distribution it has already learned to handle. It has been applied to the visual gap by randomizing rendering [56] and to the dynamics gap by randomizing physical parameters such as mass and friction [57]. Randomization can also be closed into a loop: system-identification methods adapt the simulator's parameter distribution from a small amount of real experience, so that simulated and real behaviour are matched rather than merely broadened [58], while domain-adaptation methods instead learn a

mapping that reconciles simulated and real observations [59]. A complementary strategy is to raise simulator fidelity directly; the open urban driving simulator CARLA is a widely used example [60], and chapter 6 describes a geographically faithful simulator built for this work. Actuation delay is a particularly damaging instance of the dynamics gap for articulated reversing, and is addressed both by the delay-aware learning methods discussed above and by the Smith-predictor compensator of chapter 6. That chapter characterizes the specific gaps encountered on the physical platform, namely actuation delay, hitch-angle sensing noise, and path-distribution mismatch, and addresses them through targeted engineering compensation while noting domain randomization as a route to further robustness.

## 2.6 Safety and Standards

A controller intended for a physical vehicle must ultimately answer to safety requirements, and the learning-based setting raises safety questions of its own. Safe reinforcement learning studies how to learn while respecting constraints or avoiding catastrophic states [61], and control barrier functions offer a complementary, model-based mechanism to overlay hard safety constraints on a learned policy, a direction identified here as future work [62]. For an articulated vehicle the binding safety concern is the jackknife condition, which the reward must discourage and the operational limits of chapter 6 enforce. A fielded deployment would in addition have to sit within the established automotive-safety framework, most directly the Safety of the Intended Functionality standard for hazards that arise under nominal operation [63], alongside the broader functional-safety, safety-case, and automation-level standards [64–66]. These requirements bound the present simulation study rather than being evaluated within it, and they frame the deployment reported in chapter 6.

## 2.7 Synthesis and Research Gap

The literature reviewed in this chapter divides into a mature model-based tradition and a fast-moving learning-based one, and the gap this thesis addresses lies between them. Classical control offers guarantees but demands an accurate model, careful tuning, and a separate planning stage. Learning-based control removes those demands but must contend with reward design, the reverse instability, high-dimensional perception, and the reality gap. To our knowledge, prior learned controllers for articulated vehicles have been demonstrated almost exclusively in simulation and on a single, hand-specified observation modality: the backing controllers of Nguyen and Widrow [32] and Bejar and Morán [33], and the tracking agent of Kang et al. [34], each learn from a fixed state representation and report results in simulation, without a physical deployment or a comparison across

perception modalities. This thesis addresses that combination directly. It develops a learned articulated-vehicle controller with high-dimensional perception, compares observation modalities and reward formulations through controlled ablations, benchmarks the result against tuned classical baselines, and deploys the trained policies on physical hardware with explicit attention to hitch stability and sim-to-real transfer. The remainder of the thesis carries out this programme, beginning with the vehicle model and simulation environment in chapter 3.

# Chapter 3

# Modeling, Simulation, and Problem Formulation

## 3.1 Tractor Dynamics

The tractor is represented by a planar single-track bicycle model [67], in which the front axle is lumped into one steerable wheel and the rear axle into one fixed wheel. Its global position $(x, y)$ follows from the body-frame longitudinal and lateral velocities $(v_x, v_y)$ and the heading $\psi$:

$$\dot{x} = v_x \cos\psi - v_y \sin\psi \tag{3.1}$$

$$\dot{y} = v_x \sin\psi + v_y \cos\psi \tag{3.2}$$

Lateral force is generated at each tire as a function of its slip angle. Under the small-angle assumptions appropriate to the operating envelope, the front and rear slip angles are

$$\alpha_f = \delta - \frac{v_y + l_f\dot{\psi}}{v_x}, \qquad \alpha_r = -\frac{v_y - l_r\dot{\psi}}{v_x}, \tag{3.3}$$

where $\delta$ is the steering angle. Rather than evaluating the full Pacejka magic-formula tire curve, the simulator operates in its linear region: for the modest slip angles encountered here the curve is well approximated by a constant cornering stiffness, giving the lateral tire forces

$$F_{yf} = C_f\,\alpha_f, \qquad F_{yr} = C_r\,\alpha_r. \tag{3.4}$$

This linearized-Pacejka (constant cornering-stiffness) model is the one implemented in the simulator; it preserves the lateral behaviour relevant to path tracking while avoiding the hard-to-identify parameters of the full nonlinear tire curve. Collecting the longitudinal, lateral, and yaw-moment balances, with a quadratic aerodynamic drag $F_{\text{drag}} = \frac{1}{2}C_d\rho A v_x^2$ and a net drive force $F_x$, yields the tractor dynamics. Although the drag force is negligible at low speeds, it was added as a damping force so that the simulation eventually came to rest under no input. Active braking is achieved through applying a negative drive force.

$$m\left(\dot{v}_x - v_y\dot{\psi}\right) = F_x - F_{\text{drag}} - F_{yf}\sin\delta, \tag{3.5}$$

$$m\left(\dot{v}_y + v_x\dot{\psi}\right) = F_{yf}\cos\delta + F_{yr}, \tag{3.6}$$

$$I_z\,\ddot{\psi} = l_f F_{yf}\cos\delta - l_r F_{yr}. \tag{3.7}$$

For control design and for the time-stepping used in simulation, these equations are linearized about straight-line motion ($\delta \approx 0$) at a constant reference speed $v_x = v$. With state $\boldsymbol{\xi} = [\,v_x,\ v_y,\ \dot{\psi}\,]^\top$ and input $\mathbf{u} = [\,F_x,\ \delta\,]^\top$, the linearized dynamics take the time-invariant form $\dot{\boldsymbol{\xi}} = A\boldsymbol{\xi} + B\mathbf{u}$ with

$$A = \begin{bmatrix} -\dfrac{C_d\rho A v}{2m} & 0 & 0 \\ 0 & -\dfrac{C_f + C_r}{mv} & \dfrac{l_r C_r - l_f C_f}{mv} \\ 0 & \dfrac{l_r C_r - l_f C_f}{I_z v} & -\dfrac{l_f^2 C_f + l_r^2 C_r}{I_z v} \end{bmatrix}, \qquad B = \begin{bmatrix} \dfrac{1}{m} & 0 \\ 0 & \dfrac{C_f}{m} \\ 0 & \dfrac{l_f C_f}{I_z} \end{bmatrix}. \tag{3.8}$$

The continuous system is converted to the discrete update $\boldsymbol{\xi}_{k+1} = A_d\boldsymbol{\xi}_k + B_d\mathbf{u}_k$ by a zero-order-hold discretization at the simulation step $\Delta t = 0.1\,\text{s}$, and it is this discrete map that is advanced at each timestep.

This level of fidelity is sufficient for the control tasks studied in this thesis, where the quantities of interest are path-relative position error, heading error, steering angle, and hitch articulation. The aim is a physically meaningful environment for training and evaluating classical and trained policies, not a high-fidelity vehicle simulator. An overly simplified kinematic model of the tractor would make the simulation task unrealistically easy and could reduce transferability to physical systems by neglecting actuator response, steering-rate limits, and tire-force effects on the vehicle's motion. In contrast, a kinematic model was chosen for the trailer because, at the low speeds considered in this thesis, the trailer's dominant behaviour is governed by its rolling constraint rather than by lateral tire-slip or inertial dynamics. The full-length trailer therefore primarily contributes

geometric articulation through the hitch: its axle constrains the trailer to roll along its longitudinal direction, and this nonholonomic constraint determines the evolution of the hitch angle during forward and reverse manoeuvres. This reduced-order representation preserves the main source of reverse instability while avoiding the additional tire, load-transfer, and yaw-inertia parameters required by a full dynamic trailer model.

## 3.2 Trailer Articulation and Reverse Instability

The articulated system adds a trailer body coupled to the tractor through a revolute hitch. The relevant geometry is:

- $L_h$: signed distance from the tractor rear axle to the hitch point (in both the training model and the hardware platform the hitch sits at the rear axle, so $L_h \approx 0$);
- $L_t$: distance from the hitch to the trailer axle, i.e. the trailer wheelbase;
- $\gamma = \psi - \psi_t$: the articulation angle between the tractor heading $\psi$ and the trailer heading $\psi_t$ (the same convention used by the deployment in chapter 6).

The hitch point is carried rigidly by the tractor,

$$x_h = x - L_h \cos\psi, \qquad y_h = y - L_h \sin\psi, \tag{3.9}$$

and translates with the tractor's longitudinal speed $v$. Imposing a pure-rolling (no lateral slip) constraint on the trailer axle fixes the trailer yaw rate from the lateral component of the hitch velocity:

$$\dot{\psi}_t = \frac{v}{L_t}\sin(\psi - \psi_t) = \frac{v}{L_t}\sin\gamma. \tag{3.10}$$

The articulation angle therefore evolves as

$$\dot{\gamma} = \dot{\psi} - \frac{v}{L_t}\sin\gamma, \tag{3.11}$$

where the tractor yaw rate $\dot{\psi}$ is supplied by the tractor model of eq. (3.8) (equivalently $\dot{\psi} = (v/L)\tan\delta$ for a purely kinematic tractor of wheelbase $L = l_f + l_r$). The trailer axle pose then follows from the hitch point and the trailer heading,

$$x_t = x_h - L_t \cos\psi_t, \qquad y_t = y_h - L_t \sin\psi_t. \tag{3.12}$$

The sign of $v$ in eq. (3.11) is what makes reverse motion qualitatively harder. Linearizing about the aligned configuration ($\gamma \approx 0$, so $\sin\gamma \approx \gamma$) and holding the steering-driven tractor yaw fixed, the unforced articulation mode obeys

$$\dot{\gamma} \approx -\frac{v}{L_t}\gamma, \qquad \lambda = -\frac{v}{L_t}. \tag{3.13}$$

In forward motion ($v > 0$) the eigenvalue $\lambda$ is negative: articulation errors decay and the trailer self-aligns behind the tractor. In reverse ($v < 0$) the eigenvalue becomes positive and the same errors grow exponentially toward the jackknife condition. This is an open-loop instability rather than a mere tuning difficulty, and it is the defining feature of reverse articulated manoeuvring, the reason the reverse task demands active hitch stabilization, trailer-referenced error metrics, and the dedicated reward and curriculum treatments developed in chapter 4.

The jackknife toward which this instability drives is a folded configuration from which no steering input can recover the trailer's alignment, so successful reverse manoeuvring demands anticipatory control that corrects a developing articulation error before it grows large enough to become unrecoverable. In practice the controller must regulate three coupled objectives at once (trailer path tracking, tractor placement, and articulation stability), and a controller that attends only to the tractor's cross-track error can look reasonable in the short term while driving the hitch toward instability.

## 3.3 Vehicle Parameters

The vehicle model is parameterized to represent a shunt truck (terminal tractor, or yard hostler), the class of vehicle that performs the confined low-speed trailer manoeuvring in distribution centres that this thesis targets. The specific reference vehicle is the Kalmar Ottawa T2 4x2 terminal tractor. Its layout maps directly onto the single-track bicycle model of eq. (3.8), and its manufacturer specification sheet provides genuine primary-source data [68]. The tractor has a wheelbase of 2.95 m, a maximum steering angle of 50° (0.87 rad), and is governed to a top speed near 20 m/s, more than enough to operate in the low-speed regime studied here. Table 3.1 lists the parameter set used in simulation, corresponding to the bobtail (no-trailer) diesel configuration.

Table 3.1: Vehicle parameters used in simulation, for the bobtail Kalmar Ottawa T2 diesel terminal tractor.

| Parameter | Value | Parameter | Value |
|---|---|---|---|
| $m$ | 6577 kg | $l_f$ | 1.33 m |
| $I_z$ | 16 000 kg m$^2$ | $l_r$ | 1.62 m |
| $C_f$ | 190 000 N/rad | $C_d$ | 0.8 |
| $C_r$ | 200 000 N/rad | $A$ | 7.0 m$^2$ |
| $\Delta t$ | 0.1 s | $\rho$ | 1.225 kg/m$^3$ |

The mass, wheelbase, and overall dimensions are taken directly from the manufacturer specification sheet. The yaw moment of inertia $I_z$ is estimated from the mass and body envelope using a standard moment-of-inertia estimator for road vehicles [69]. The cornering stiffnesses $C_f$ and $C_r$ are derived from published heavy-truck tire data [70], scaled by the number of tires per axle (a single pair at the steered front axle against dual tires at the drive axle) and by the axle loads. The front and rear stiffnesses therefore differ, unlike the equal-stiffness simplification commonly used for passenger-car examples. The longitudinal centre-of-gravity split is estimated from an approximate static front/rear axle load distribution, and the aerodynamic terms $C_d$ and $A$ are engineering estimates for a terminal-tractor cab. Neither the aerodynamic terms nor the centre-of-gravity split is backed by primary data, but their influence is minor in the operating regime of interest: aerodynamic forces are negligible at governed yard speeds, and the centre-of-gravity split mainly shapes high-speed transient response rather than the low-speed trajectories studied here. The timestep $\Delta t = 0.1\,\mathrm{s}$ corresponds to the 10 Hz control loop used in the hardware deployment (chapter 6), so the simulation and the deployed controller operate at a matched rate.

The values in table 3.1 describe the bobtail tractor, which defines the dynamic tractor subsystem used in simulation. The trailer is represented kinematically through the hitch model of eq. (3.11), for the reasons given in section 3.1. It is not treated as massless, however: its dominant inertial contribution is retained as an equivalent longitudinal load acting through the hitch, preserving the effect of trailer mass on longitudinal acceleration and control effort.

## 3.4 Simulation Environment

The training and evaluation environment is a custom 2D simulation built on the Farama Foundation Gymnasium library [71]. Gymnasium provides a standardized API for environments to integrate directly with reinforcement learning algorithm libraries, including Stable Baselines3 [72].

The simulation is implemented in Python, with Pygame [73] providing collision masks and rendering. It was built around a rendering pipeline that draws the tractor-trailer and lane geometry as a top-down frame; this frame can be viewed live for monitoring, or passed directly to the image encoder as a bird's-eye-view (BEV) observation, making high-dimensional perception available to the agent without a separate image-synthesis step. At each timestep, the discrete-time tractor-trailer state-space equations are re-evaluated from the previous state and control inputs, updating the vehicle position, heading, hitch angle, and associated error signals, and Pygame redraws the scene as a 2D top-down view from which the BEV image observation is cropped. For large-scale training this rendered observation is later produced without the Pygame renderer by the batched

GPU simulator (section 4.9), but the rendering pipeline remains the conceptual origin of the BEV observation and the reference for what it encodes.

The simulation environment includes:

1. **Vehicle Model**: A combined dynamic tractor (bicycle model with a linearized Pacejka tire model [74]) and kinematic trailer with hitch constraints, using the shunt-truck parameters of section 3.3.
2. **Lane Generation**: Randomly generated lane paths using cubic spline interpolation, producing varying curvature paths for training and evaluation.
3. **Obstacle Placement and Local Replanning**: Randomly placed static obstacles within the lane and a local path planner that perturbs the centreline around those obstacles for avoidance experiments.
4. **Collision Detection**: Pygame sprite mask-based collision detection, providing pixel-precise contact sensing between the vehicle body and obstacles or lane boundaries.
5. **Observations**: the rendered top-down BEV image described above, alongside engineered state and range-sensor (lidar) observations; the full set of observation modalities available to the policy is defined in chapter 4.

## 3.5 Environment Tasks

The environment implementation is structured around reusable task and observation components. A base lane-following task is combined with different observation heads, while obstacle-aware variants reuse the same task logic through a mixin that adds obstacle generation, local path planning, and obstacle-aware reward evaluation. This avoids maintaining separate forward, reverse, lidar, and BEV implementations with duplicated task logic.

### 3.5.1 Line Following

The lane-following environment extends the base tractor-trailer environment to present a lane-tracking task. A reference lane is generated at the start of each episode using cubic spline interpolation through randomly placed waypoints. The reward function encourages the agent to keep both the tractor and trailer within the lane boundaries while maintaining progress along the path.

Reverse line-following environments use the same path generation and rendering pipeline, but reverse the longitudinal command and compute reward and tracking terms

with the trailer as the leading unit, so that the same task can be studied in backward manoeuvres without changing the task definition.

### 3.5.2 Obstacle Avoidance

The obstacle avoidance environment adds static obstacles within the lane. The agent must navigate the tractor-trailer through the lane while avoiding obstacles, which requires both path-following behaviour and collision avoidance. Instead of following the global lane centreline directly, the environment constructs a local path around the placed obstacles and evaluates tracking error relative to this locally planned path. Obstacles are placed as a single cluster of one or two circular obstacles per episode, drawn from a small set of layout categories (clear, shift, squeeze, and blocked), so that the difficulty distribution is controlled and can be stratified during evaluation. The locally planned avoidance path is used only to shape the reward and is never exposed to the agent, so the detour must be inferred from the lidar or BEV observation. In addition to terminal collision penalties, the base environment applies a per-step proximity penalty near blocked occupancy-grid cells, so both lane boundaries and inserted obstacles discourage the agent before contact occurs. Forward and reverse obstacle environments are both available. The scalar difficulty rating that indexes these layouts, and the reward that keys on it, are developed in section 4.3.1.

### 3.5.3 Actions and Episode Termination

Longitudinal speed is not a control degree of freedom in this work. It is held at a fixed operating value (5 m/s forward, $-5$ m/s in reverse), so every method solves the same lateral problem and forward and reverse tasks share a common control interface. The agent instead commands a two-dimensional action, a continuous steering rate $\dot{\delta}$ and a stop signal $\sigma$:

$$\mathbf{a} = [\dot{\delta},\, \sigma] \in [-25^\circ/\mathrm{s},\, 25^\circ/\mathrm{s}] \times [-1,\, 1]. \tag{3.14}$$

Commanding the steering rate rather than the steering angle reflects the fact that the steering angle cannot change instantaneously on a physical vehicle, so regulating its rate of change is more physically realistic. The stop signal is a go/stop decision: when $\sigma$ exceeds a fixed threshold the vehicle halts and the episode terminates, which lets the policy decline to attempt a layout it judges impassable. This stop-capable action is used across all learning-based tasks, so that the interface does not change between the no-obstacle and obstacle settings; discrete algorithms realise the same choice through a joint steering-and-stop action set. Stopping is not universally desirable, however: in the obstacle-free tasks a stop is treated as a failure and penalized, and it earns reward only on difficult obstacle layouts. The difficulty-keyed stop-gate reward that governs this trade-off is developed in section 4.3.1, and the experimental protocols in chapter 5.

Episodes terminate on collision, jackknife ($|\gamma| > 90°$), or successful path completion.

# Chapter 4

# Learning Framework: Observation and Representation Design, Reward Formulation, and Policy Training

## 4.1 Learning Framework Overview

*[Section content omitted for review.]*

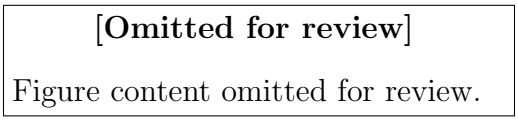



Figure 4.1: Architecture of the learning system: an observation (engineered state, lidar, or BEV image) is encoded into a feature vector and passed to the TD3 actor and critics, which output a steering rate and a stop signal; the reward combines path-tracking and hitch-stability terms.

*[Section content omitted for review.]*

## 4.2 Reinforcement Learning with TD3

*[Section content omitted for review.]*

[equation omitted for review] (4.1)

*[Section content omitted for review.]*

## 4.3 Reward Formulation

[equation omitted for review] (4.2)

[equation omitted for review] (4.3)

[equation omitted for review] (4.4)

[equation omitted for review] (4.5)

[equation omitted for review] (4.6)

[equation omitted for review] (4.7)

[equation omitted for review] (4.8)

[equation omitted for review] (4.9)

[equation omitted for review] (4.10)

[equation omitted for review] (4.11)

[equation omitted for review] (4.12)

[equation omitted for review] (4.13)

[equation omitted for review] (4.14)

[equation omitted for review] (4.15)

[equation omitted for review] (4.16)

[equation omitted for review] (4.17)

[equation omitted for review] (4.18)

[equation omitted for review] (4.19)

[equation omitted for review] (4.20)

[equation omitted for review] (4.21)

[equation omitted for review] (4.22)

*[Section content omitted for review.]*

### 4.3.1 Obstacle-Navigation Reward Design

*[Section content omitted for review.]*

## 4.4 Hyperparameter Optimization

*[Section content omitted for review.]*

## 4.5 Perception and Representation Design

*[Section content omitted for review.]*

### 4.5.1 Lidar Observation

*[Section content omitted for review.]*

### 4.5.2 Bird's-Eye-View Observation

*[Section content omitted for review.]*

### 4.5.3 Image Encoding and Representation Learning

*[Section content omitted for review.]*

[equation omitted for review] (4.23)

[equation omitted for review] (4.24)

[equation omitted for review] (4.25)

[equation omitted for review] (4.26)

*[Section content omitted for review.]*

### 4.5.4 Observation Space Summary

*[Section content omitted for review.]*

Table 4.1: Observation space components

| **[Omitted for review]** |
| --- |
| Table content omitted for review. |

## 4.6 Training High-Dimensional-Observation Policies

*[Section content omitted for review.]*

### 4.6.1 An Off-Policy Instability Hypothesis

*[Section content omitted for review.]*

### 4.6.2 Route A: Pretraining the Representation

*[Section content omitted for review.]*

### 4.6.3 Route B: Imitation Warm-Start

*[Section content omitted for review.]*

### 4.6.4 Route C: Geometry Distillation

*[Section content omitted for review.]*

### 4.6.5 Safe Adaptation

*[Section content omitted for review.]*

### 4.6.6 Curriculum and Training Protocol

*[Section content omitted for review.]*

## 4.7 Failure Replay Curriculum

*[Section content omitted for review.]*

## 4.8 Evaluation Methodology

*[Section content omitted for review.]*

[equation omitted for review] (4.27)

*[Section content omitted for review.]*

## 4.9 Scalable Simulation: GPU-Parallel Batched Environments

*[Section content omitted for review.]*

### 4.9.1 Batched Environment Design

*[Section content omitted for review.]*

### 4.9.2 Numerical Parity

*[Section content omitted for review.]*

### 4.9.3 Throughput and Scaling

*[Section content omitted for review.]*

Table 4.2: Environment-stepping throughput (transitions per second) for the batched GPU environment versus the single-core scalar reference, measured on an RTX 4080 SUPER.

| **[Omitted for review]** |
| --- |
| Table content omitted for review. |

*[Section content omitted for review.]*

Table 4.3: Same batched code, NumPy (CPU) versus CuPy (GPU), by batch size $N$ and observation type. Throughput in transitions per second; speedup is CuPy relative to NumPy.

| **[Omitted for review]** |
| --- |
| Table content omitted for review. |

*[Section content omitted for review.]*

# Chapter 5

# Results and Discussion

## 5.1 Path Following Performance

*[Section content omitted for review.]*

Table 5.1: Planned run matrix for the experimental campaign

| [**Omitted for review**] |
| --- |
| Table content omitted for review. |

## 5.2 Observation Space Ablation

*[Section content omitted for review.]*

| [**Omitted for review**] |
| --- |
| Figure content omitted for review. |

Figure 5.1: Learning curves (mean episode return ± 1 s.d.) for forward lane-following: observation space ablation. Shaded bands show one standard deviation across evaluation episodes at each checkpoint.

| [**Omitted for review**] |
| --- |
| Figure content omitted for review. |

Figure 5.2: Learning curves for reverse lane-following: observation space ablation.

Table 5.2: Forward lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).

**[Omitted for review]**

Table content omitted for review.

Table 5.3: Reverse lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).

**[Omitted for review]**

Table content omitted for review.

## 5.3 Lidar Beam-Count Ablation

*[Section content omitted for review.]*

Table 5.4: Lidar beam-count ablation on forward lane-following (dense reward, no obstacles, 30 eval episodes).

**[Omitted for review]**

Table content omitted for review.

## 5.4 Reward Formulation Ablation

*[Section content omitted for review.]*

**[Omitted for review]**

Figure content omitted for review.

Figure 5.3: Learning curves for forward lane-following: reward formulation ablation.

**[Omitted for review]**

Figure content omitted for review.

Figure 5.4: Learning curves for reverse lane-following: reward formulation ablation.

Table 5.5: Forward lane-following: reward formulation ablation (lidar-24 obs, no obstacles, 30 eval episodes).

**[Omitted for review]**

Table content omitted for review.

Table 5.6: Reverse lane-following: reward formulation ablation (lidar-24 obs, no obstacles, 30 eval episodes).

**[Omitted for review]**

Table content omitted for review.

## 5.5 Statistical Rigor and Generalization

*[Section content omitted for review.]*

## 5.6 Training Strategy: Failure Replay Curriculum

*[Section content omitted for review.]*

**[Omitted for review]**

Figure content omitted for review.

Figure 5.5: Learning curves for reverse lane-following: failure replay curriculum vs random reset baseline. Both agents use identical hyperparameters, observation space, and reward formulation.

Table 5.7: Failure replay curriculum ablation on reverse lane-following (lidar-24 obs, multiplicative reward, 30 held-out eval episodes).

**[Omitted for review]**

Table content omitted for review.

## 5.7 Reverse Maneuvering Success

*[Section content omitted for review.]*

## 5.8 Perception Robustness

*[Section content omitted for review.]*

## 5.9 Obstacle Navigation

*[Section content omitted for review.]*

Table 5.8: Forward obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).

| [Omitted for review] |
| --- |
| Table content omitted for review. |

Table 5.9: Reverse obstacle-avoidance: TD3 performance by observation space (multiplicative reward, 5–10 obstacles per episode, 30 eval episodes).

| [Omitted for review] |
| --- |
| Table content omitted for review. |

## 5.10 Benchmark Comparison

Table 5.10: Forward lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).

| [Omitted for review] |
| --- |
| Table content omitted for review. |

Table 5.11: Reverse lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).

| [Omitted for review] |
| --- |
| Table content omitted for review. |

*[Section content omitted for review.]*

### 5.10.1 Information Parity Between Learned and Classical Controllers

*[Section content omitted for review.]*

Table 5.12: Reference-path information available to each method at a single control step on the no-obstacle tasks. The learned policy's observation is a strict superset of the information used by the pure-pursuit and PID baselines; only MPC receives an explicit preview of the path ahead.

| [Omitted for review] |
| --- |
| Table content omitted for review. |

*[Section content omitted for review.]*

## 5.11 Failure Analysis and Deployment Relevance

*[Section content omitted for review.]*

Table 5.13: Recommended experiment matrix for the thesis

| **[Omitted for review]** |
| --- |
| Table content omitted for review. |

# Chapter 6

# Sim-to-Real Deployment

## 6.1 Overview

The preceding chapters developed and evaluated an articulated tractor-trailer controller entirely within a 2D Pygame/Gymnasium simulation environment. This chapter describes the transfer of that controller to a physical robotic platform, constituting the sim-to-real component of the thesis. Unlike a conventional model-based stack, the deployed system does not re-implement a hand-derived tracking law on the vehicle; instead, the simulation-trained reinforcement-learning (RL) policies are executed directly on hardware through a thin bridging layer that reproduces, in real time, the observation and action interfaces the policies were trained against. The objective is to demonstrate that the vehicle model, the observation definitions, and the learned control logic established in simulation can be ported to real hardware with minimal algorithmic redesign, and to characterize the engineering challenges encountered during that transfer.

The deployment is built on a deliberately stripped-down Autoware stack. The standard Autoware localization and perception pipelines are retained essentially unchanged, while the planning and control layers are replaced by a custom RL bridge that runs the forward and reverse policies. Two pieces of supporting infrastructure are described in detail because they are specific to the articulated, sensor-light platform: an onboard LiDAR-based hitch-angle estimator that recovers the articulation angle without any trailer-mounted sensor, and a Smith predictor [42] that compensates for actuator delay so that delay-free-trained policies behave correctly on a lagging physical actuator.

## 6.2 Hardware Platform

### 6.2.1 Tractor Vehicle

The physical tractor is an Agilex Hunter SE, a compact electric wheeled unmanned ground vehicle with an Ackermann steering geometry. The relevant mechanical parameters are a

wheelbase of 0.65 m, a wheel radius of 0.15 m, and a maximum steering angle of 0.436 rad (25°). Vehicle actuation is commanded over a CAN bus on a 20 ms cycle; speed commands are encoded as 16-bit signed integers in units of 1 mm/s (the commanded speed in m/s scaled by 1000) and steering commands as 16-bit signed integers scaled by approximately 1320 counts per radian ($\approx 0.8$ mrad per count).

### 6.2.2 Trailer

The trailer is coupled to the tractor through a revolute hitch joint located at (or marginally behind) the tractor rear axle, matching the training-time trailer model in which the hitch coincides with the rear axle. The lab trailer has a wheelbase of approximately 2 m and a body width of roughly 0.33 m[1]. The articulation angle $\gamma$, defined as the difference between the tractor and trailer headings ($\gamma = \psi_{\text{truck}} - \psi_{\text{trailer}}$), is *estimated onboard* from the tractor's own LiDAR rather than measured by any trailer-mounted sensor; the procedure is detailed in Section 6.3.4. The estimate is published to the ROS2 network on the `/vehicle/trailer_state` topic; the control bridge consumes the most recent published value, with no staleness gating currently applied. Two distinct angular limits appear in the stack and should not be conflated: the onboard estimator clamps the *published* articulation estimate to $|\gamma| \leq 90°$, whereas the deployment trailer model defines a tighter operational limit of $|\gamma| \leq 80°$ (1.40 rad), itself below the 90° jackknife condition used to terminate simulation episodes.

### 6.2.3 Sensor Suite

The vehicle is instrumented with the following sensors:

- **LiDAR**: RoboSense Helios mechanical LiDAR mounted at the front of the tractor, publishing 3D point clouds on `/rslidar_points`. This single sensor is reused for three purposes: NDT-based localization [83], environment perception, and onboard hitch-angle estimation.
- **Cameras**: Six Basler Pylon industrial cameras (rear, rear-right, centre, left, merging, and right) producing compressed colour images. The cameras are not consumed by the LiDAR-based RL policies and are used for logging and monitoring only.
- **IMU**: WitMotion 6-DoF inertial measurement unit, with a gyroscope-bias estimator applied in the localization pipeline.

[1]The trailer wheelbase is recorded in the deployment configuration as a nominal 2 m pending an exact in-person measurement; controller and estimator parameters are not sensitive to small errors in this value.

- **GNSS**: u-blox ZED-F9P receiver with RTK corrections, providing centimetre-level position estimates converted to MGRS coordinates for Autoware localization.
- **Wheel odometry**: Vehicle speed and steering-angle feedback recovered from CAN messages and fused with IMU data for dead-reckoning between GNSS and scan-matching updates.

## 6.3 Software Architecture

### 6.3.1 Middleware and Framework

The deployment stack runs on ROS2 Humble (Ubuntu 22.04) [84] and is integrated into the Autoware autonomous-driving framework [85]. Autoware provides modular localization, perception, planning, and control pipelines whose interfaces are defined by standardized ROS2 message types. As introduced in the overview, this deployment keeps the standard sensing, localization, and perception modules and replaces only the planning and control layers with the learned RL bridge.

### 6.3.2 Relationship to the Conventional Articulated Pipeline

For context, Autoware's conventional approach to articulated vehicles is fully model-based: a truck-trailer mode manager routes goals to a freespace planner (a hybrid $A^*$ search [15] that respects the non-holonomic constraints of the articulated vehicle), and the resulting reference trajectory is tracked by a model-predictive or LQR lateral controller. This model-based pipeline exists within the wider project and represents the standard, well-understood baseline for trailer control; the relevant control background is reviewed in Chapter 2. The contribution of *this* thesis, however, is the learned controller, and the deployment described here therefore routes control through the RL bridge instead of the planner/MPC chain. The two are mutually exclusive at run time and are selected at launch.

### 6.3.3 RL Bridge Node Architecture

The learned control path comprises the following principal nodes:

`lane_reference_node` Loads the Lanelet2 map, selects the active lane from the ego odometry and the current goal, and publishes the lane centreline as a reference path together with a `drive_enabled` flag and a `drive_direction` flag (forward or reverse). This node supplies the path geometry against which the policy's tracking observation is computed.

`rl_bridge_node` The core of the deployment. It loads the trained forward and reverse TD3 policies (Stable-Baselines3 [72] / PyTorch [86] checkpoints) and runs a 10 Hz control loop. On each tick it assembles the policy observation in exactly the layout used during training, an 8-dimensional state vector (steering angle, articulation angle, tractor and trailer tracking errors, and look-ahead path curvature) concatenated with a 24-beam LiDAR range vector, queries the active policy deterministically, and converts the 2-dimensional action (steering rate and commanded speed) into the steering-angle and acceleration commands expected by the vehicle interface. Direction-dependent post-processing (action smoothing and kinematic rate scaling) is applied for the reverse policy, whose outputs are inherently more oscillatory than the forward policy's.

`hitch_angle_estimator` Estimates the articulation angle from the tractor LiDAR and publishes it on `/vehicle/trailer_state` (Section 6.3.4).

`initialpose_shim` Bridges the RViz `/initialpose` message to the Autoware pose-initializer service, allowing the operator to seed localization interactively.

`controller_canbridge` Translates `autoware_control_msgs/Control` commands into Hunter SE CAN frames and parses vehicle feedback (velocity, steering angle) back into ROS2 topics.

The forward and reverse policies are loaded from separate checkpoints and selected according to the `drive_direction` flag, so that the same bridge handles both manoeuvring regimes without retraining or reconfiguration.

### 6.3.4 Hitch Angle Estimation

Because the trailer carries no articulation encoder and no trailer-mounted sensing, the articulation angle is recovered geometrically from the tractor's front LiDAR, which observes the front (and, at larger angles, the side) face of the trailer. The estimator operates on each incoming point cloud as follows.

**Region of interest.** Points are first cropped to a box in the LiDAR frame that brackets the expected location of the trailer face. The lateral extent of this region is shifted to track the face as the trailer swings: using the current filtered estimate $\gamma$, the lateral offset is set to

$$\Delta y = -\,D_{\text{face}}\,\sin\gamma, \tag{6.1}$$

where $D_{\text{face}}$ is the approximate longitudinal distance from the hitch pivot to the face. This keeps the cropped points on the trailer face across the full range of articulation rather than only near $\gamma = 0$.

**Rectangle fitting.** Rather than fitting a single planar normal, the estimator fits the orientation of the trailer body to the cropped points using the minimum-closeness rectangle criterion of Zhang *et al.* [87]. For a candidate orientation $\theta$, the points are rotated into axis-aligned coordinates $(u, v)$ and scored by their closeness to the nearest edges of the bounding rectangle,

$$C(\theta) = \sum_i \min\big(d_i^u,\, d_i^v\big)^2, \qquad d_i^u = \min\big(|u_i - u_{\min}|,\, |u_i - u_{\max}|\big), \tag{6.2}$$

with $d_i^v$ defined analogously. The fitted orientation minimizes $C(\theta)$ augmented by a dimension-prior penalty that favours rectangles consistent with the known trailer width and length:

$$\theta^\star = \arg\min_{\theta \in [0,\pi)} \big[\, C(\theta) + \lambda\, P_{\dim}(\theta) \,\big]. \tag{6.3}$$

This formulation is robust to whether only the front face, both the front and a side face (an "L" shape), or only a side face is visible, the dominant cause of failure for a naive single-face normal estimate at large articulation. The four 90° heading candidates implied by $\theta^\star$ are disambiguated by choosing the one closest to the previous filtered estimate and most consistent with the dimension prior, yielding a trailer longitudinal direction $\hat{\mathbf{d}}_t = (\cos\theta^\star, \sin\theta^\star)$.

**Articulation angle and filtering.** The raw articulation angle is the signed angle between the truck-forward axis $\hat{\mathbf{x}} = (1, 0)$ and $\hat{\mathbf{d}}_t$,

$$\tilde{\gamma} = \operatorname{atan2}\big(\hat{x}_1 \hat{d}_{t,2} - \hat{x}_2 \hat{d}_{t,1},\; \hat{\mathbf{x}} \cdot \hat{\mathbf{d}}_t\big), \tag{6.4}$$

to which a field-calibration sign $s$ and offset $\gamma_0$ are applied and the result clamped to the operational range, $\gamma = \operatorname{clip}(s\,\tilde{\gamma} + \gamma_0,\, -\gamma_{\max},\, \gamma_{\max})$. The sign convention is chosen so that the published value matches Autoware's $\gamma = \psi_{\text{truck}} - \psi_{\text{trailer}}$. The raw per-scan estimate is then passed through a constant-angular-velocity Kalman filter [88] on the state $[\gamma,\, \dot{\gamma}]$, with Mahalanobis innovation gating to reject outlier scans, and a short moving-average is maintained in parallel as a diagnostic. The filtered angle and its rate populate the `hitch_angle` and `hitch_rate` fields of the published `TrailerState`.

This onboard, geometry-plus-filter estimator is the actual mechanism behind the `/vehicle/trailer_state` signal; it is not an external measurement. Its noise characteristics, and the way they are handled, are revisited in the gap analysis (Section 6.4.2).

### 6.3.5 Trailer State Estimation

Because the trailer carries no independent localization, its full pose is reconstructed from the tractor's localized kinematic state and the estimated articulation angle. Given the

tractor's rear-axle position and heading ($\mathbf{p}_{\text{truck}}$, $\psi_{\text{truck}}$) from `/localization/kinematic_state`, the hitch-point position is

$$\mathbf{p}_{\text{hitch}} = \mathbf{p}_{\text{truck}} + d_h \begin{bmatrix} \cos\psi_{\text{truck}} \\ \sin\psi_{\text{truck}} \end{bmatrix}, \tag{6.5}$$

where $d_h$ is the (small, near-zero) signed hitch offset relative to the tractor rear axle. The trailer heading follows directly from the articulation angle, $\psi_{\text{trailer}} = \psi_{\text{truck}} - \gamma$, and the trailer rear axle is located at

$$\mathbf{p}_{\text{trailer}} = \mathbf{p}_{\text{hitch}} - L_t \begin{bmatrix} \cos\psi_{\text{trailer}} \\ \sin\psi_{\text{trailer}} \end{bmatrix}, \tag{6.6}$$

where $L_t \approx 2\,\text{m}$ is the trailer wheelbase. This reconstruction mirrors the hitch-kinematics model derived in Chapter 3, confirming that the simulation formulation is directly reusable in the deployment stack, and it provides the trailer-referenced tracking errors that form part of the policy observation.

### 6.3.6 Actuator-Delay Compensation: Smith Predictor

The simulation trains the policies against an idealized actuator that responds instantaneously to commands, whereas the physical steering and drive systems exhibit a first-order lag of the order of 50–150 ms. A policy trained for instantaneous response will, when confronted with a delayed actuator, repeatedly over-correct and excite a lag-induced oscillation. To reconcile the two, a Smith-predictor-style compensator is implemented that lets the delay-free-trained policy act on a delay-compensated state estimate.

The compensator maintains two "shadow" instances of the same kinematic `StateSpaceTractorTrai` model used in simulation: a *delay-free* shadow with zero actuation time constant, and a *delayed* shadow whose time constant matches the measured hardware lag. On each control tick both shadows are advanced with the command currently propagating through the actuator pipeline. The discrepancy between the measured vehicle state and the delayed shadow forms a residual,

$$\mathbf{r}_k = \mathbf{x}_k^{\text{meas}} - \mathbf{x}_k^{\text{delayed}}, \tag{6.7}$$

which captures model mismatch beyond pure delay (integration scheme, ground-contact effects, friction). The state presented to the policy is the delay-free shadow corrected by this residual,

$$\hat{\mathbf{x}}_k = \mathbf{x}_k^{\text{free}} + \mathbf{r}_k, \tag{6.8}$$

so that the policy "sees" an effectively delay-free vehicle while the residual keeps the prediction anchored to reality. The shadows are re-synchronized to the measured state on reset and whenever a large state jump is detected. The compensator is implemented

and available but is disabled by default, as the shadow time constants require per-platform tuning before the loop can be closed in anger; combined with the explicit steering-rate limit it forms the primary mitigation for the actuation gap discussed in Section 6.4.1.

**Learned dynamics model (in progress).** The Smith predictor's fidelity is bounded by the accuracy of the analytical kinematic shadow. Work is in progress to replace, or augment, that shadow with a neural-network approximation of the true system dynamics. A data pipeline converts recorded driving logs (rosbags) into time-aligned 10 Hz state–action–next-state datasets (deriving the trailer pose from the truck pose and the estimated hitch angle where it is not directly recorded) and a multilayer-perceptron dynamics model (`DynamicsMLP`) is trained on these datasets. The learned model is evaluated both by one-step prediction error against held-out logs and by closed-loop rollout divergence from the kinematic model, the latter serving to quantify and localize the sim-to-real dynamics gap. This component is not yet part of the live control loop and is reported here as ongoing work rather than a validated result.

## 6.4 Sim-to-Real Gap Analysis

Transferring a simulation-trained policy to physical hardware introduces discrepancies between the simulated and real-world dynamics. The following gap sources were identified and addressed during the deployment process.

### 6.4.1 Actuation Dynamics

The simulation model assumes instantaneous steering response, whereas the physical vehicle exhibits a first-order steering lag introduced by the CAN command latency and the Hunter SE's internal steering servo. This is the highest-impact discrepancy for a policy trained on instantaneous actuation. It is mitigated by the Smith-predictor compensator of Section 6.3.6, which presents a delay-compensated state to the policy, and by a steering-rate limit applied to the commanded output, consistent with the $\dot{\delta}_{\max}$ constraint used during simulation training.

### 6.4.2 Hitch-Angle Sensing

In simulation, the articulation angle $\gamma$ is available exactly from the vehicle state. On hardware it is estimated from LiDAR returns and is therefore subject to noise from sparse or partial returns on the trailer face, mis-segmentation of the region of interest, and momentary loss of the face at large articulation. These error modes are addressed within the estimator itself rather than by a downstream filter: the rectangle-fitting formulation (Equation 6.3) is robust to partial face visibility, and the constant-angular-velocity

Kalman filter with innovation gating rejects outlier scans and smooths the published signal. If the trailer face is lost entirely the estimator withholds new estimates, and the bridge continues from the last published value until the face is reacquired; no automatic disengagement on stale data is currently implemented.

### 6.4.3 Path Distribution

The policies are trained on smooth, spline-generated reference paths, whereas the deployment reference is the centreline of a Lanelet2 lane, which is piecewise-linear and may contain comparatively sharp vertices. Re-sampling the centreline does not fully reconcile the difference in path-curvature distribution between training and deployment, and this mismatch is a contributing factor to any residual tracking error observed on hardware. It is noted here as a characterization of the gap rather than something fully eliminated in the current deployment.

### 6.4.4 Jackknife Recovery

The simulation introduced a terminal episode condition when $|\gamma| > 90°$ but provided no explicit recovery mechanism, since the episode simply resets. Rather than adding a separate runtime supervisor on the vehicle, jackknife resilience is instilled during *training*, through two complementary measures. First, a recovery-scenario reset distribution spawns a fraction of reverse episodes already in the near-jackknife region (articulation magnitudes of roughly 0.35–0.80 rad, i.e. 20–45°), so the policy is repeatedly exposed to high-articulation states its nominal training distribution rarely visits and must learn to steer the trailer back toward alignment. Second, an anti-jackknife override is applied *within the training environment*: whenever the articulation exceeds approximately 0.7 rad (40°), the policy's steering command is replaced by a maximum counter-steer that drives the articulation back toward zero. This prevents the policy from learning that oscillatory, near-jackknife behaviour is survivable, since every dangerous excursion is corrected during training and chattering no longer pays off. No dedicated runtime recovery override is currently part of the deployed bridge; on hardware, jackknife avoidance therefore relies on the behaviour learned through these measures together with the operational $|\gamma| \leq 80°$ limit.

### 6.4.5 Localization Dependency

The simulation provides ground-truth vehicle pose at every step. The hardware stack instead depends on the Autoware localization pipeline, which fuses RTK-GNSS, gyro-based odometry, and LiDAR NDT scan matching through an extended Kalman filter. GNSS outages or localization divergence therefore represent a failure mode that does not

exist in simulation; the current deployment treats localization failure as an emergency-stop condition.

## 6.5 High-Fidelity Geographic Simulation

The 2D Pygame environment of chapter 3 is deliberately abstract: it captures the tractor-trailer kinematics and lane geometry but not the three-dimensional and photometric structure that the deployed perception and localization stack actually consumes. A complementary, higher-fidelity simulation environment was therefore built to exercise the full Autoware stack (LiDAR-based NDT localization, perception, and the RL bridge) inside a geographically faithful "digital twin" of a real operating site as a complement to the physical trials.

This environment is built on the Gazebo simulator [89], extending the open-source Rudis Laboratories georeferenced world-generation toolchain [90]. Real locations are reconstructed as textured 3D meshes authored in Blender [91], whose physically based materials are baked into texture maps for efficient real-time rendering. Building on that toolchain, the target distribution-centre site was rebuilt in Blender directly from public geographic data, with both layers imported through the Blender GIS plugin: building footprints were pulled from OpenStreetMap [92] and extruded to form the 3D building meshes, and these were draped over aerial orthoimagery from the ESRI World Imagery basemap, replacing the lower-resolution ground textures of the original toolchain with the higher-resolution ESRI imagery (fig. 6.1). Because the reconstruction is driven entirely by the OpenStreetMap footprints and the ESRI imagery layer, the same procedure generates a Gazebo world for any real distribution centre from its coordinates alone, rather than being tied to a single hand-authored site. Each world is anchored to GIS coordinates through the SDF `spherical_coordinates` element, which fixes the simulation origin to a real latitude, longitude, and elevation; the templated world generator additionally computes the sun's azimuth and elevation from the site coordinates and a chosen UTC date and time, so that lighting and shadows match the real location and time of day. The `electrans` tractor and trailer models are then spawned into these worlds, so that the same articulated vehicle can be driven through a virtual replica of the target environment (figs. 6.2 and 6.3).

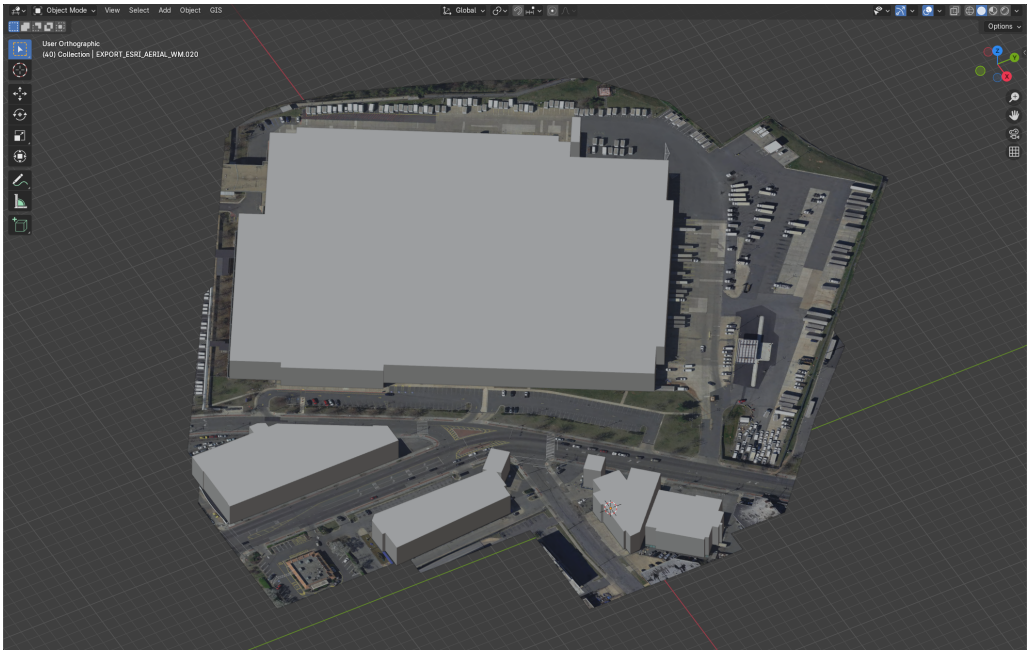


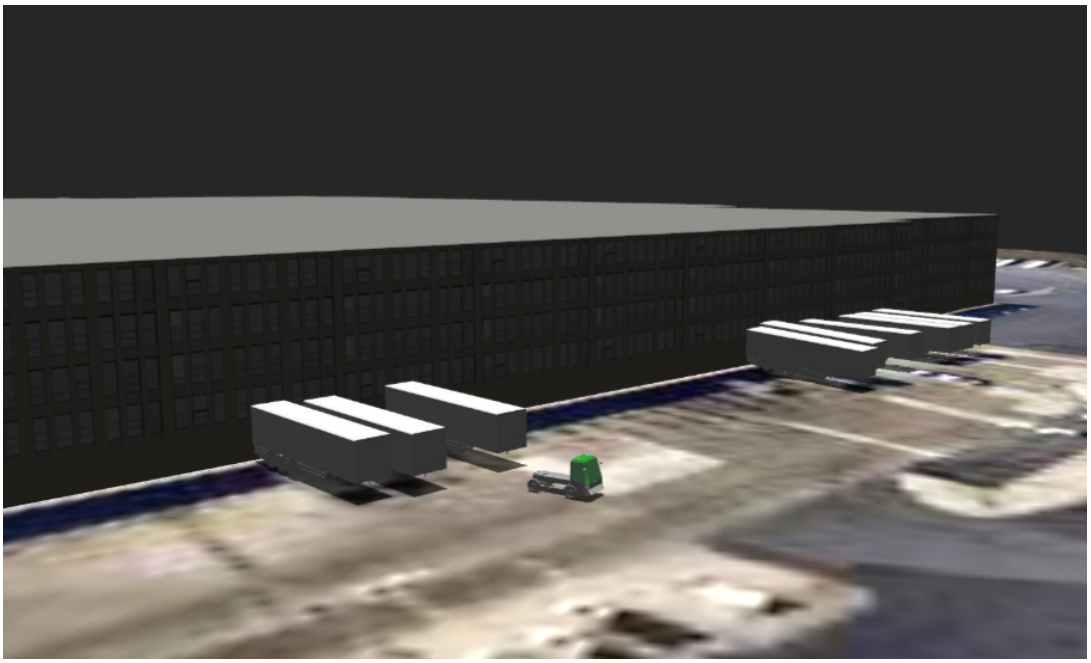

Figure 6.1: Reconstruction of the target distribution-centre site in Blender. OpenStreetMap building footprints are extruded to form the 3D building meshes and draped over high-resolution aerial orthoimagery from the ESRI World Imagery basemap, both layers imported through the Blender GIS plugin. The same GIS-driven procedure reconstructs any real distribution centre from its geographic coordinates.

**Figure unavailable**

Reconstructed distribution centre exported to a georeferenced Gazebo world.

`figures/gazebo/gazebo_dc_world.png`

(a) The reconstructed site exported to a georeferenced Gazebo world.



(b) The `electrans` tractor-trailer spawned at a loading dock within the world.

Figure 6.2: The georeferenced site running in Gazebo. The exported world (a) preserves the real building geometry and aerial ground texture, and the `electrans` articulated vehicle (b) is spawned into it so that the same policies evaluated in simulation can be exercised against the full Autoware sensing and localization stack.

Because the world is georeferenced and rendered in three dimensions, this environment directly targets several of the sim-to-real gaps catalogued above. A LiDAR sensor model produces point clouds against real building and ground geometry, so NDT scan matching and the onboard hitch-angle estimator (section 6.3.4) can be exercised against realistic returns; the geographic anchoring lets the GNSS and map frames be reproduced consistently with the physical site; and the photorealistic rendering makes it possible to evaluate camera-based perception that the abstract 2D environment cannot represent. It is also the natural setting in which to prototype the infrastructure sensor nodes of section 6.6, since virtual LiDAR units can be placed at arbitrary surveyed locations in the georeferenced world. The environment and its GIS-driven world-generation pipeline are complete; what remains is the quantitative characterization of policy behaviour within it, which is identified as future work in chapter 7.

### 6.5.1 Articulated Vehicle Model

The articulated vehicle spawned into the georeferenced worlds is the `electrans` tractor-trailer, shown in fig. 6.3. It is defined in the conventional ROS/Gazebo manner as a URDF assembled from `xacro` macros, rather than as a hand-written SDF model. The tractor shares its entire `xacro` description with the Agilex Hunter model used elsewhere in the project: the same parameterized macros define an Ackermann steering geometry (a pair of revolute front-steering links bounded by the maximum steering angle), continuous-rotation rear wheels coupled by a mimic joint, the link inertials, and the Gazebo friction and `ros2_control` blocks. The two vehicles differ only in scale and in the body mesh: the Hunter's compact chassis mesh is replaced by a cab-over terminal-tractor body, and the model is scaled accordingly. Reusing a single `xacro` description in this way keeps the simulated tractor kinematically consistent with the physical Hunter platform on which the policies are deployed, while presenting the larger visual form of the terminal tractor this thesis targets.

The trailer is coupled to the tractor through a revolute hitch joint at the tractor rear axle, matching the hitch convention of the kinematic model in chapter 3. It is modelled as a single box body carried on one axle and reuses the same wheel meshes and inertial macros as the tractor, so that the complete articulated system is expressed through the shared macro library. Because the model is authored as standard xacro, it drops directly into both the Gazebo digital twin of this section and the wider Autoware simulation tooling without modification.

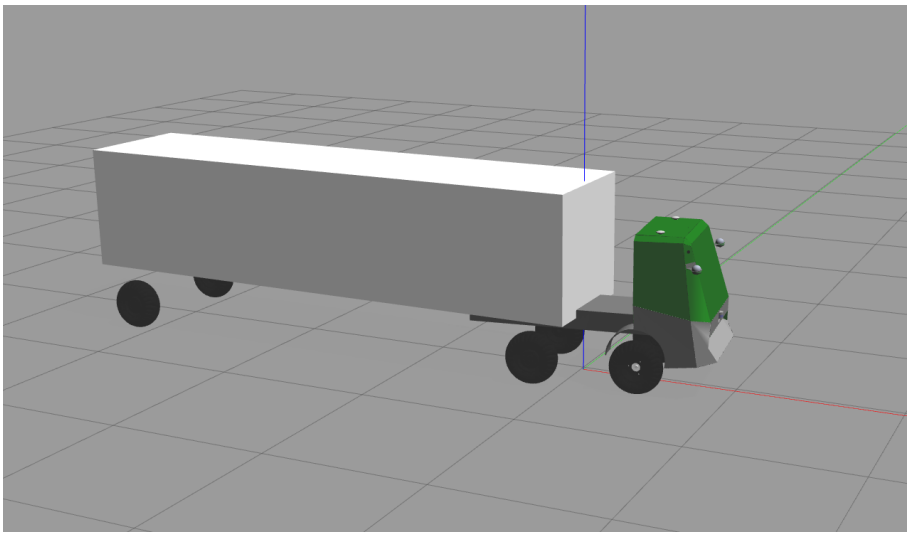


Figure 6.3: The `electrans` tractor-trailer model. The tractor reuses the Agilex Hunter `xacro` description (Ackermann front steering, continuous rear wheels, and `ros2_control` blocks), rescaled and given a cab-over terminal-tractor body mesh; the box trailer is attached through a revolute hitch joint at the tractor rear axle.

## 6.6 Infrastructure Sensor Nodes

A structural limitation of the onboard sensing arrangement is that the trailer occludes the region directly behind it. The tractor-mounted LiDAR cannot see past the trailer, and instrumenting the trailer itself is undesirable: it would add cabling, power, calibration, and a wireless link to an otherwise passive, interchangeable body, and would have to be repeated for every trailer. This is most acute during reverse manoeuvring, precisely when visibility of the space behind the trailer matters most.

This limitation is addressed with *off-vehicle infrastructure sensor nodes*: fixed LiDAR units placed at surveyed vantage points around the operating area, each providing a bird's-eye view of obstacles (crucially including the region behind the trailer) without any sensor being mounted on the trailer. Each node is calibrated into a shared global frame so that its detections can be expressed in the same coordinates as the onboard perception output and fused into Autoware's perception and occupancy-grid representation, yielding a persistent, occlusion-resilient view of the obstacles around the articulated vehicle that complements the limited onboard field of view.

This capability is implemented and running in the laboratory as the indoor-perception stack. Each infrastructure node runs Euclidean clustering on its own LiDAR returns to detect candidate objects. An offboard fusion pipeline transforms every node's detections into the shared global frame, merges detections that refer to the same physical object across nodes by their overlapping footprint polygons, and time-aligns the streams against each node's modelled latency before passing the merged detections to an extended-Kalman-filter multi-object tracker. The tracker emits a single stable, deduplicated obstacle list for the whole monitored area.

Because the laboratory sensor nodes and their perception stack run under ROS1 (Noetic) while the vehicle runs under ROS2 (Humble), the two are joined by a dedicated interface node: it subscribes to the ROS1 perception outputs and republishes the fused obstacle list over ROS2, where it is consumed by the vehicle's Autoware perception and occupancy-grid representation and thus reaches the RL bridge. Critically, this interface node runs *offboard* on the laboratory compute rather than on the vehicle. The vehicle's onboard Jetson therefore runs only ROS2, and never has to host a dual ROS1/ROS2 stack; the ROS1 dependency is confined entirely to the off-vehicle infrastructure, which both simplifies the embedded software image and keeps the version-sensitive ROS1 perception packages off the resource-constrained onboard computer.

In laboratory operation the infrastructure nodes recover the region directly behind the trailer that the onboard LiDAR cannot observe, extending obstacle coverage to [TODO]% of the vehicle's immediate surroundings during reverse manoeuvres, at an added end-to-end latency of [TODO] ms relative to onboard-only perception.

**Figure unavailable**

Diagram of the infrastructure LiDAR sensor nodes providing a bird's-eye view of obstacles behind the trailer.

`figures/system_architecture/infrastructure_sensor_nodes`

Figure 6.4: Infrastructure sensor-node architecture: fixed off-vehicle LiDAR units give a bird's-eye view of obstacles around the articulated vehicle, covering the region behind the trailer that the onboard sensors cannot observe. Their detections are fused and tracked offboard and delivered to the vehicle over ROS2, so the onboard computer runs only ROS2.

## 6.7 Deployment Results

Hardware validation trials were conducted on [TODO: describe test location and scenario]. The forward policy achieved [TODO: describe forward lane-following behaviour], with a mean trailer cross-track error of [TODO] m and articulation maintained within $|\gamma| < [TODO]^\circ$ throughout nominal operation. Reverse manoeuvring [TODO: describe outcome]. [TODO: report the effect of enabling the Smith predictor on oscillation and tracking once tuned.]

## 6.8 Summary

This chapter demonstrated that the tractor-trailer control architecture developed in simulation can be ported to a physical platform by executing the trained RL policies directly through a bridging layer, rather than by re-deriving an analytical controller on the vehicle. The deployment retains Autoware's standard localization and perception while replacing planning and control with an RL bridge that runs the forward and reverse policies at 10 Hz over the same observation and action interfaces used in training. The principal engineering contributions are this RL bridge and its integration into a stripped-down Autoware stack, the onboard LiDAR hitch-angle estimator that recovers the articulation angle by rectangle fitting and Kalman filtering without any trailer-mounted sensor, and the Smith-predictor compensator (with an in-progress learned dynamics model) that reconciles delay-free-trained policies with a lagging physical actuator. The gap analysis confirms that the simulation captures the dominant dynamics relevant to controller design, while identifying actuation delay, hitch-angle sensing, path-distribution mismatch, and localization reliability as the primary points of divergence between the simulated and physical systems. Finally, the planned infrastructure sensor nodes are introduced as a route to occlusion-resilient perception behind the trailer without adding sensors to the trailer itself.

# Chapter 7

# Conclusion

## 7.1 Summary of Findings

*[Section content omitted for review.]*

## 7.2 Limitations

*[Section content omitted for review.]*

## 7.3 Future Work

*[Section content omitted for review.]*

# Appendix A

# Classical Controller Implementation

The deep reinforcement-learning policy is benchmarked against four classical path-tracking controllers. This appendix documents their implementation: the exact control laws, the forward and reverse variants, the way each controller was tuned, and the harness used to evaluate them. The four controllers are chosen to span the classical control spectrum, from a purely geometric law through to constrained optimal control with preview:

- **Pure Pursuit** — a geometric proportional law;
- **PID** — classical error feedback with an integral term;
- **LQR** — optimal static linear feedback on the error dynamics;
- **Kinematic LTV MPC** — optimal constrained control with preview.

All four are implemented in the batched simulator (`tractor_trailer_rl_cupy`) so that they run inside the same vectorized environment as the learned policies. Crucially, every controller emits the same action as the RL agent — a bounded *steering rate* — so the comparison between classical and learned control is made on identical terms.

## A.1 Common framework

### A.1.1 Observation layout and shared action mapping

Each controller is a function of the observation vector already produced by the environment. For the trailer tasks this vector is

$$o = [\, s,\ \gamma,\ e_y,\ e_\psi,\ e_{y,t},\ e_{\psi,t},\ \kappa,\ \ldots\,], \tag{A.1}$$

where $s$ is the current steering angle, $\gamma$ the hitch (articulation) angle, and the error pairs $(e_y, e_\psi)$ and $(e_{y,t}, e_{\psi,t})$ are the cross-track and heading errors of the tractor and of the

trailer respectively; $\kappa$ is the reference path curvature at the lookahead point. The trailing entries carry additional task-specific channels (for example lidar returns) that the classical controllers do not use.

Every controller computes a desired steering angle $\delta_{\text{des}}$ and then converts it to the environment's steering-rate action through a common saturating map,

$$\dot{\delta} = \text{clip}\left(\frac{\delta_{\text{des}} - s}{\Delta t},\ -\dot{\delta}_{\max},\ +\dot{\delta}_{\max}\right), \tag{A.2}$$

where $\Delta t$ is the simulation step and $\dot{\delta}_{\max}$ the steering-rate limit. Because the learned policy also outputs $\dot{\delta}$, and eq. (A.2) applies to all four baselines identically, the classical and learned controllers act through exactly the same interface.

### A.1.2 Forward and reverse tracking

Each controller has a forward and a reverse variant, reflecting a structural asymmetry of the articulated vehicle. Driving *forward*, the tractor leads and the trailer follows passively, so it suffices to regulate the tractor errors $(e_y, e_\psi)$ towards zero. Driving *in reverse*, the trailer leads and the coupling inverts: the hitch angle $\gamma$ becomes *open-loop unstable* (small articulation errors grow and the vehicle jackknifes), so the reverse controllers regulate the trailer errors $(e_{y,t}, e_{\psi,t})$ and must additionally include an explicit hitch-stabilizing action. This asymmetry is why the reverse variants below are not simply the forward laws with a sign change, and why their gains had to be tuned separately (appendix A.6).

### A.1.3 Vehicle parameters

The benchmark uses the full-size shunt-truck configuration. The parameters that enter the controllers are summarized in table A.1; here $L_1$ is the tractor wheelbase, $L_2$ the effective trailer length (hitch to trailer axle), and $V$ the fixed longitudinal speed held constant throughout the fixed-speed path-tracking experiments. The large length ratio $L_2/L_1 \approx 4.2$ is what makes the reverse hitch dynamics difficult to stabilize.

Table A.1: Shunt-truck parameters used by the classical controllers.

| Symbol | Quantity | Value |
|---|---|---|
| $L_1$ | Tractor wheelbase | 2.95 m |
| $L_2$ | Trailer length (hitch to axle) | 12.5 m |
| $V$ | Fixed longitudinal speed | 5.0 m/s |
| $\Delta t$ | Simulation step | 0.1 s |
| $\delta_{\max}$ | Steering-angle limit | 0.87 rad ($\approx 50°$) |
| $\dot{\delta}_{\max}$ | Steering-rate limit | 25°/s |

In the reverse variants the speed enters with a sign, $V < 0$. Because the environment's tractor yaw response to steering does not itself flip sign in reverse, the along-path kinematics use the magnitude $|V|$ while the yaw dynamics retain the signed $V$; this distinction is carried consistently through the LQR and MPC models below.

## A.2 Pure Pursuit

The pure-pursuit controller [18] is a fixed proportional geometric law. Forward, it combines a curvature feed-forward with proportional cross-track and heading feedback on the tractor errors,

$$\delta_{\text{des}} = -\big(k_{\text{ff}} \arctan(L_1 \kappa) + k_y\, e_y + k_\theta\, e_\psi\big). \tag{A.3}$$

Reverse, it regulates the trailer errors and adds an explicit hitch-stabilizing term $k_{\text{hitch}}\, \gamma$ to counter the open-loop-unstable articulation,

$$\delta_{\text{des}} = k_{\text{hitch}}\, \gamma + k_{\text{ff}}\, \kappa + k_y\, e_{y,t} + k_\theta\, e_{\psi,t}. \tag{A.4}$$

Both laws are stateless functions of the observation, so the controller is fully vectorized across the parallel environments. Besides serving as a benchmark, the tuned pure-pursuit controller doubles as the reference ("teacher") signal for the guided reward and as a feasibility oracle: a path that a well-tuned pure-pursuit controller can complete is deemed geometrically feasible.

## A.3 PID

The PID controller extends the proportional law with an integral term that removes steady-state cross-track offset. The integral is accumulated per environment and clamped to prevent windup, and it is reset at episode boundaries. Forward,

$$\delta_{\text{des}} = -k_{\text{ff}}\, \kappa - K_p\, e_y - K_i \int e_y - K_d\, e_\psi + K_{p,t}\, e_{y,t}, \tag{A.5}$$

and reverse (regulating the trailer errors, with the hitch stabiliser),

$$\delta_{\text{des}} = k_{\text{hitch}}\, \gamma + k_{\text{ff}}\, \kappa + K_p\, e_{y,t} + K_i \int e_{y,t} + K_d\, e_{\psi,t}. \tag{A.6}$$

The derivative gain $K_d$ multiplies the heading error, so it acts as a heading-damping term rather than a numerical time derivative. The integrator uses the anti-windup update

$$I \leftarrow \text{clip}\big(I + e\, \Delta t,\ -I_{\max},\ +I_{\max}\big), \tag{A.7}$$

where $e$ is the regulated cross-track error ($e_y$ forward, $e_{y,t}$ reverse). Because the controller carries this per-environment state, the evaluation loop passes a termination mask so that $I$ is zeroed for any environment that resets on the current step.

## A.4 LQR

The LQR baseline is the canonical optimal-linear controller: static feedback $u = -Kx$ on a linear model of the tracking-error dynamics, plus a curvature feed-forward [67]. The error state is

$$x = [\, e_{\text{lat}},\ e_{\text{head}},\ \gamma \,]^\top, \tag{A.8}$$

the lateral and heading error of the leading unit (tractor forward, trailer reverse) together with the hitch angle; the input is the steering angle $u = \delta$. The continuous error dynamics $\dot{x} = A_c x + B_c u$ are direction-specific. Forward,

$$A_c = \begin{bmatrix} 0 & V & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -\dfrac{V}{L_2} \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ \dfrac{V}{L_1} \\ \dfrac{V}{L_1} \end{bmatrix}, \tag{A.9}$$

and reverse, where the leading trailer progresses along the path at speed $|V|$ while the yaw dynamics keep the signed $V$,

$$A_c = \begin{bmatrix} 0 & |V| & 0 \\ 0 & 0 & \dfrac{V}{L_2} \\ 0 & 0 & -\dfrac{V}{L_2} \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ 0 \\ \dfrac{|V|}{L_1} \end{bmatrix}. \tag{A.10}$$

The pair $(A_c, B_c)$ is discretized by zero-order hold at the step $\Delta t$ to give $(A_d, B_d)$. With state and input weights $Q = \text{diag}(q_{\text{lat}}, q_{\text{head}}, q_\gamma)$ and $R$, the infinite-horizon gain follows from the discrete algebraic Riccati equation. Its stabilizing solution $P$ yields

$$K = \left(R + B_d^\top P B_d\right)^{-1} B_d^\top P A_d. \tag{A.11}$$

Because the speed is constant, $K$ is computed once at construction and then applied to every environment, so the controller stays fully vectorized. The command adds a steady-state feed-forward that holds the vehicle on a constant-curvature arc: a feed-forward steer $\delta_{\text{ff}} = L_1 \kappa$ and a hitch reference $\gamma_{\text{ref}} = \text{sign}(V)\, L_2\, \kappa$, giving

$$\delta = \delta_{\text{ff}} - K\,(x - x_{\text{ref}}), \qquad x_{\text{ref}} = [\, 0,\ 0,\ \gamma_{\text{ref}} \,]^\top, \tag{A.12}$$

which is then saturated to $\pm\delta_{\max}$ and mapped to a steering rate through eq. (A.2). The reverse weights place a heavy penalty on control effort and hitch deviation, which is what damps the otherwise unstable reverse plant.

## A.5 Kinematic LTV MPC

The final baseline is a linear time-varying model-predictive controller [6] that adds a finite preview horizon and explicit input constraints. The MPC uses the same error state $x = [\, e_{\text{lat}}, e_{\text{head}}, \gamma \,]^\top$ as the LQR but plans over an $N$-step horizon.

### A.5.1 Prediction models

Forward, the tractor leads, so the MPC tracks the tractor errors with the steering angle $\delta$ as the input, giving the same $(A_c, B_c)$ as the forward LQR model (A.9). Reverse, servoing the trailer pose with the unstable hitch dynamics inside the optimization causes the long shunt trailer to oscillate and jackknife. The reverse controller therefore uses a flatness / virtual-input reformulation (following the Autoware trailer LTV-MPC): the QP plans the *trailer* as a stable bicycle driven by a *virtual* trailer steer $\delta_T$, and the hitch enters only through an inner loop that has been closed to the stable first-order dynamics $\dot{\gamma} = K(\delta_T - \gamma)$,

$$A_c = \begin{bmatrix} 0 & |V| & 0 \\ 0 & 0 & \dfrac{V}{L_2} \\ 0 & 0 & -K \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ 0 \\ K \end{bmatrix}. \tag{A.13}$$

The reference curvature enters both models as a measured disturbance through $E_c = [\, 0,\ -|V|,\ 0 \,]^\top$. After the QP returns the virtual input, an analytic inner map realises it on the physical tractor and supplies the anti-jackknife counter-steer,

$$\delta_f = \arctan\!\left( \frac{L_1}{L_2}\, \operatorname{sign}(V)\, \sin\gamma + \frac{L_1 K}{|V|}\, (\delta_T - \gamma) \right). \tag{A.14}$$

The $1/|V|$ factor is what flips the feedback sign in reverse and produces the stabilizing counter-steer.

### A.5.2 Condensed quadratic program

The continuous models are discretized by zero-order hold and the state trajectory is condensed onto the input sequence, so that the optimization variable is the vector of $N$ future steering inputs. With the stacked weight $\bar{Q} = I_N \otimes Q$, a control penalty $R$, an input-rate penalty $R_d$, and the first-difference operator $T = I - I^{(-1)}$ (so that $T$ forms

$\Delta u$), the cost Hessian is

$$H = 2\big(B_u^\top \bar{Q}\, B_u + R\, I + R_d\, T^\top T\big), \tag{A.15}$$

where $B_u$ is the condensed input-to-state map. The problem is subject to a steering-rate box on the increments, $|\Delta u| \le \dot{\delta}_{\max}\, \Delta t$, and a steering box $|u| \le \delta_{\max}$. It is solved per environment with OSQP. The rate-penalty term $R_d\, T^\top T$ is weighted heavily in reverse so the controller does not chase cross-track measurement ripple. The overall per-step procedure is summarized in algorithm 1.

**Algorithm 1** Kinematic LTV MPC step (per environment)

1: **Precompute (once):** discretize $(A_c, B_c, E_c)$ by ZOH; build the condensed maps $A_x, B_u, E_w$ and the constant Hessian $H$ from (A.15).
2: **On episode reset:** clear the warm-start input $u_{\text{prev}} \leftarrow 0$.
3: Read $x_0 = [\, e_{\text{lat}}, e_{\text{head}}, \gamma \,]$ and curvature $\kappa$ from the observation.
4: Form the linear cost term from $x_0$, the previewed curvature disturbance, and $u_{\text{prev}}$.
5: Solve the QP with OSQP subject to the steering-rate and steering boxes.
6: **if** solver fails **then** $u \leftarrow u_{\text{prev}}$ ▷ safe fallback
7: **else** $u \leftarrow$ first element of the optimal input sequence
8: **end if**
9: $u_{\text{prev}} \leftarrow u$
10: $\delta \leftarrow u$ (forward) **or** apply inner map (A.14) to obtain $\delta_f$ (reverse).
11: **return** steering rate from (A.2).

Only the first input of the optimal sequence is applied (receding horizon); the solved input is retained as the warm start for the next step and, like the PID integrator, is reset at episode boundaries. If the solver fails to converge, the controller falls back to the previous command.

## A.6 Gain-tuning methodology

The controllers above are defined by their gains and weights: the proportional and feed-forward gains of pure pursuit, the PID gains, the LQR state/input weights, and the MPC horizon and cost weights. Rather than report a specific set of numbers, which are particular to one vehicle configuration, the point of interest is the procedure used to obtain them, which is the same for every controller.

Each controller is tuned by a bounded random search (a gain sweep) run directly against the simulator [93]. On each iteration a candidate gain set is sampled uniformly from hand-set ranges, the controller is rolled out over a batch of parallel environments,

and the candidate is scored by the lexicographic key

$$\big(\text{completion rate},\ -\,\text{mean cross-track error}\big), \tag{A.16}$$

so that reliably completing the path is maximized first and, among gains that complete, the one with the tightest tracking is preferred. The best candidate is retained. This search is run *separately for each controller and each direction*, because the forward and reverse plants differ so strongly.

The reverse variants in particular could not simply inherit the gains tuned on the small laboratory vehicle: on the shunt truck those gains destabilize the vehicle, and several gains (notably the hitch-stabilizing terms) change sign. This is a direct consequence of the open-loop-unstable reverse hitch dynamics and the large trailer-to-tractor length ratio $L_2/L_1 \approx 4.2$ discussed in appendix A.1.2; the reverse controllers were therefore re-tuned from scratch for the target geometry.

## A.7 Evaluation harness

All four controllers are evaluated by a single harness that reuses the same batched environment as the reinforcement-learning policies, so that the classical baselines and the learned policy are measured on identical paths under identical dynamics. The harness runs both driving directions and, for each controller, records three per-episode metrics:

- **Completion** — the fraction of episodes that reach the goal (the mean success flag);
- **Trailer cross-track error** — the mean absolute lateral error of the trailer over the episode, a measure of tracking accuracy;
- **Peak hitch angle** — the largest articulation angle reached, a measure of how close the vehicle came to jackknifing (the jackknife threshold is 1.4 rad).

The harness handles the stateful controllers by passing a per-environment termination mask into the controller at each step, so the PID integrator (A.7) and the MPC warm start are reset exactly when an environment restarts. Evaluating every controller in the same loop, on the same feasible path pool, is what makes the reinforcement-learning comparison in the results chapter a fair, like-for-like one. The measured numbers are reported there rather than in this appendix.

# Appendix B

# Full Hyperparameter Tables

Table B.1: TD3 Hyperparameters

| [**Omitted for review**] |
| --- |
| Table content omitted for review. |

# Appendix C

# Additional Plots

*[Section content omitted for review.]*

# Bibliography


[1] S. D. Pendleton, H. Andersen, X. Du, X. Shen, M. Meghjani, Y. H. Eng, D. Rus, and M. H. Ang, "Perception, planning, control, and coordination for autonomous vehicles," *Machines*, vol. 5, no. 1, p. 6, 2017.

[2] P. Fancher and C. Winkler, "Directional performance issues in evaluation and design of articulated heavy vehicles," *Vehicle System Dynamics*, vol. 45, pp. 607–647, 2007.

[3] C. Altafini, A. Speranzon, and B. Wahlberg, "A feedback control scheme for reversing a truck and trailer vehicle," *IEEE Transactions on Robotics and Automation*, vol. 17, no. 6, pp. 915–922, 2001.

[4] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*. Englewood Cliffs, NJ: Prentice-Hall, 1990.

[5] B. Schnapp, "LQR control of vehicle bicycle model," Tech. Rep. ECE 682 Course Project Report, University of Waterloo, Dept. of Mechanical and Mechatronics Engineering, 2024. Unpublished graduate course report.

[6] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Madison, WI: Nob Hill Publishing, 2 ed., 2017.

[7] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[8] A. M. C. Odhams, R. L. Roebuck, B. A. Jujnovich, and D. Cebon, "Active steering of a tractor–semi-trailer," *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, vol. 225, no. 7, pp. 847–869, 2011.

[9] B. A. Jujnovich and D. Cebon, "Path-following steering control for articulated vehicles," *Journal of Dynamic Systems, Measurement, and Control*, vol. 135, no. 3, p. 031006, 2013.

[10] M. Beglini, L. Lanari, and G. Oriolo, "Anti-jackknifing control of tractor-trailer vehicles via intrinsically stable MPC," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8806–8812, 2020.

[11] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, "Flatness and defect of nonlinear systems: Introductory theory and examples," *International Journal of Control*, vol. 61, no. 6, pp. 1327–1361, 1995.

[12] C. Samson, "Control of chained systems: Application to path following and time-varying point-stabilization of mobile robots," *IEEE Transactions on Automatic Control*, vol. 40, no. 1, pp. 64–77, 1995.

[13] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[14] S. M. LaValle, *Planning Algorithms*. Cambridge, UK: Cambridge University Press, 2006.

[15] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010.

[16] O. Ljungqvist, N. Evestedt, M. Cirillo, D. Axehill, and O. Holmer, "Lattice-based motion planning for a general 2-trailer system," in *2017 IEEE Intelligent Vehicles Symposium (IV)*, pp. 819–824, 2017.

[17] R. Oliveira, O. Ljungqvist, P. F. Lima, and B. Wahlberg, "Optimization-based on-road path planning for articulated vehicles," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 15572–15579, 2020.

[18] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Tech. Rep. CMU-RI-TR-92-01, Robotics Institute, Carnegie Mellon University, 1992.

[19] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, "Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing," in *American Control Conference (ACC)*, pp. 2296–2301, 2007.

[20] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2 ed., 2018.

[21] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[22] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pp. 387–395, 2014.

[23] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2016.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 1861–1870, 2018.

[26] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 1587–1596, 2018.

[27] R. Tymkow and B. Schnapp, "Application of deep deterministic policy gradient (ddpg) and decision transformer reinforcement learning to vehicle path following," Tech. Rep. ME 780 Course Project Report, University of Waterloo, Dept. of Mechanical and Mechatronics Engineering, 2024. Unpublished graduate course report.

[28] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2022.

[29] F. Codevilla, M. Müller, A. M. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[30] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J.-M. Allen, V.-D. Lam, A. Bewley, and A. Shah, "Learning to drive in a day," in *2019 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8248–8254, 2019.

[31] A. El Sallab, M. Abdou, E. Perot, and S. Yogamani, "Deep reinforcement learning framework for autonomous driving," *Electronic Imaging, Autonomous Vehicles and Machines*, vol. 29, pp. 70–76, 2017.

[32] D. H. Nguyen and B. Widrow, "Neural networks for self-learning control systems," *IEEE Control Systems Magazine*, vol. 10, no. 3, pp. 18–23, 1990.

[33] E. Bejar and A. Morán, "Backing up control of a self-driving truck-trailer vehicle with deep reinforcement learning and fuzzy logic," in *2018 IEEE International Symposium on Signal Processing and Information Technology (ISSPIT)*, pp. 202–207, 2018.

[34] Q. Kang, A. Hartmannsgruber, S.-H. Tan, X. Zhang, and C.-M. Chew, "Deep reinforcement learning based tractor-trailer tracking control," in *2024 IEEE International Conference on Intelligent Transportation Systems (ITSC)*, pp. 3147–3153, 2024.

[35] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278–287, 1999.

[36] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 15084–15097, 2021.

[37] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pp. 41–48, 2009.

[38] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum learning for reinforcement learning domains: A framework and survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.

[39] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*, 2016.

[40] T. J. Walsh, A. Nouri, L. Li, and M. L. Littman, "Learning and planning in environments with delayed feedback," *Autonomous Agents and Multi-Agent Systems*, vol. 18, no. 1, pp. 83–105, 2009.

[41] Y. Bouteiller, S. Ramstedt, G. Beltrame, C. Pal, and J. Binas, "Reinforcement learning with random delays," in *International Conference on Learning Representations (ICLR)*, 2021.

[42] O. J. M. Smith, "Closer control of loops with dead time," *Chemical Engineering Progress*, vol. 53, no. 5, pp. 217–219, 1957.

[43] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[44] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097–1105, 2012.

[45] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[46] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[47] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d," in *European Conference on Computer Vision (ECCV)*, pp. 194–210, 2020.

[48] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.

[49] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[50] S. Lange and M. Riedmiller, "Deep auto-encoder neural networks in reinforcement learning," in *International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2010.

[51] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[52] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel, "Deep spatial autoencoders for visuomotor learning," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 512–519, 2016.

[53] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.

[54] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3d classification and segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 652–660, 2017.

[55] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737–744, 2020.

[56] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.

[57] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[58] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, "Closing the sim-to-real loop: Adapting simulation randomization with real world experience," in *2019 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8973–8979, 2019.

[59] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kelcey, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige, S. Levine, and V. Vanhoucke, "Using simulation and domain adaptation to improve efficiency of deep robotic grasping," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4243–4250, 2018.

[60] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, "CARLA: An open urban driving simulator," in *Conference on Robot Learning (CoRL)*, pp. 1–16, 2017.

[61] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.

[62] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *18th European Control Conference (ECC)*, pp. 3420–3431, 2019.

[63] International Organization for Standardization, "ISO 21448:2022 road vehicles — safety of the intended functionality." ISO Standard, 2022.

[64] International Organization for Standardization, "ISO 26262: Road vehicles — functional safety." ISO Standard, 2018.

[65] Underwriters Laboratories, "UL 4600: Standard for safety for the evaluation of autonomous products." UL Standard, 2022.

[66] SAE International, "J3016: Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles." SAE Standard, 2021.

[67] R. Rajamani, *Vehicle Dynamics and Control*. New York: Springer, 2 ed., 2011.

[68] Kalmar Ottawa, *Ottawa 4x2 DOT/EPA Terminal Tractor: Standard Specifications*. Kalmar Ottawa, 2022. Manufacturer specification sheet.

[69] D. D. MacInnis, W. E. Cliff, and K. W. Ising, "A comparison of moment of inertia estimation techniques for vehicle dynamics simulation," in *SAE Technical Paper Series*, no. 970951, (Warrendale, PA), pp. 99–117, SAE International, 1997.

[70] R. D. Ervin, C. B. Winkler, J. E. Bernard, and R. K. Gupta, "Effects of tire properties on truck and bus handling, volume IV: Final report," Tech. Rep. DOT-HS-802-143, Highway Safety Research Institute, University of Michigan, Ann Arbor, MI, 1976. Prepared for the National Highway Traffic Safety Administration, U.S. Department of Transportation.

[71] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[72] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[73] Pygame community, "pygame: Python game development library." https://www.pygame.org. Cross-platform set of Python modules built on SDL.

[74] H. B. Pacejka, *Tyre and Vehicle Dynamics*. Oxford: Butterworth-Heinemann, 3 ed., 2012.

[75] R. Liu, J. Lehman, P. Molino, F. Petroski Such, E. Frank, A. Sergeev, and J. Yosinski, "An intriguing failing of convolutional neural networks and the CoordConv solution," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[76] S. Fujimoto, D. Meger, and D. Precup, "Off-policy deep reinforcement learning without exploration," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 2052–2062, 2019.

[77] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 20132–20145, 2021.

[78] D. Yarats, A. Zhang, I. Kostrikov, B. Amos, J. Pineau, and R. Fergus, "Improving sample efficiency in model-free reinforcement learning from images," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 10674–10681, 2021.

[79] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Proceedings of the Conference on Robot Learning (CoRL)*, pp. 66–75, 2020.

[80] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[81] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in *Proceedings of Workshop on Machine Learning Systems (LearningSys) at the 31st Conference on Neural Information Processing Systems (NIPS)*, 2017.

[82] N. J. Higham, "The scaling and squaring method for the matrix exponential revisited," *SIAM Journal on Matrix Analysis and Applications*, vol. 26, no. 4, pp. 1179–1193, 2005.

[83] P. Biber and W. Straßer, "The normal distributions transform: A new approach to laser scan matching," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2743–2748, 2003.

[84] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[85] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*, pp. 287–296, 2018.

[86] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.

[87] X. Zhang, W. Xu, C. Dong, and J. M. Dolan, "Efficient L-shape fitting for vehicle detection using laser scanners," in *2017 IEEE Intelligent Vehicles Symposium (IV)*, pp. 54–59, 2017.

[88] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[89] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2149–2154, 2004.

[90] B. Perseghetti and Rudis Laboratories, "rudislabs_gazebo_worlds: Georeferenced Gazebo world models." https://github.com/rudislabs/rudislabs_gazebo_worlds, 2021. Open-source software repository, Apache-2.0 licence.

[91] Blender Online Community, *Blender — a 3D Modelling and Rendering Package*. Blender Foundation, Amsterdam. https://www.blender.org.

[92] OpenStreetMap contributors, "OpenStreetMap." https://www.openstreetmap.org, 2024. Map data available under the Open Database License (ODbL).

[93] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.